

Московский государственный технический
университет им. Н. Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”

Курс “Парадигмы и конструкции языков программирования”
Отчёт по домашнему заданию

Выполнил:
Студент группы ИУ5-31Б
Луценко Александра

Проверил:
Гапанюк Ю.Е

```

import asyncio
import json
import os
import re
from telegram import (
    Update,
    InlineKeyboardButton,
    InlineKeyboardMarkup,
    ReplyKeyboardMarkup,
    KeyboardButton
)
from telegram.ext import (
    Application,
    CommandHandler,
    CallbackQueryHandler,
    MessageHandler,
    ContextTypes,
    filters
)
from urllib.parse import quote
from typing import Dict, List, Optional

SEND_MONEY_STATES = {
    'NONE': 0,
    'WAITING_FOR_RECIPIENT': 1,
    'WAITING_FOR_AMOUNT': 2,
    'WAITING_FOR_MESSAGE': 3,
    'WAITING_FOR_CONFIRMATION': 4
}

REQUEST_MONEY_STATES = {
    'NONE': 0,
    'WAITING_FOR_RECIPIENT': 1,
    'WAITING_FOR_AMOUNT': 2,
    'WAITING_FOR_MESSAGE': 3,
    'WAITING_FOR_CONFIRMATION': 4
}

class Database:
    def __init__(self, filename="users.json"):
        self.filename = filename
        self.users = self.load_data()
        print(f"DEBUG: Загружено {len(self.users)} пользователей")
        for chat_id, user in self.users.items():
            print(f"DEBUG: Пользователь {chat_id}: {user}")

```

```

def load_data(self):
    try:
        if os.path.exists(self.filename):
            with open(self.filename, 'r', encoding='utf-8') as f:
                data = json.load(f)
                return {int(k): v for k, v in data.items()}
        return {}
    except Exception as e:
        print(f"Ошибка загрузки данных: {e}")
        return {}

def save_data(self):
    try:
        with open(self.filename, 'w', encoding='utf-8') as f:
            json.dump({str(k): v for k, v in self.users.items()}, f, ensure_ascii=False,
indent=2)
        return True
    except Exception as e:
        print(f"Ошибка сохранения данных: {e}")
        return False

def is_user_registered(self, chat_id: int) -> bool:
    result = chat_id in self.users
    print(f"DEBUG: Проверка регистрации {chat_id}: {result}")
    return result

def is_user_registered_by_username(self, username: str) -> bool:
    search_username_with_at = f"@{username}" if not username.startswith('@')
else username
    search_username_without_at = username[1:] if username.startswith('@') else
username

    result = any(
        user.get('username') == search_username_with_at or
        user.get('username') == search_username_without_at
        for user in self.users.values()
    )
    print(f"DEBUG: Проверка username '{username}': {result}")
    return result

def get_chat_id_by_username(self, username: str) -> Optional[int]:
    search_username_with_at = f"@{username}" if not username.startswith('@')
else username

```

```
        search_username_without_at = username[1:] if username.startswith('@') else
username
```

```
        for chat_id, user in self.users.items():
            user_username = user.get('username', "")
            if user_username == search_username_with_at or user_username ==
search_username_without_at:
                print(f"DEBUG: Найден chat_id {chat_id} для username '{username}'")
                return chat_id
```

```
        print(f"DEBUG: Не найден chat_id для username '{username}'")
        return None
```

```
def save_user(self, chat_id: int, username: str, name: str, phone: str):
    if not username.startswith('@'):
        username = f"@{username}"
```

```
    self.users[chat_id] = {
        'username': username,
        'name': name,
        'phone': phone
    }
    print(f"DEBUG: Сохранен пользователь {chat_id}: {self.users[chat_id]}")
    return self.save_data()
```

```
def get_user(self, chat_id: int):
    return self.users.get(chat_id)
```

```
def get_all_users(self):
    return self.users
```

```
class MoneyBot:
```

```
    def __init__(self, token: str):
        self.application = Application.builder().token(token).build()
        self.db = Database()
```

```
        self.send_money_data: Dict[int, dict] = {}
        self.request_money_data: Dict[int, dict] = {}
        self.user_phones: Dict[int, str] = {}
```

```
        self.setup_handlers()
```

```
def setup_handlers(self):
    self.application.add_handler(CommandHandler("start", self.start_command))
    self.application.add_handler(CommandHandler("help", self.help_command))
```

```

        self.application.add_handler(CommandHandler("debug",
self.debug_command))
        self.application.add_handler(CallbackQueryHandler(self.callback_handler))
        self.application.add_handler(MessageHandler(filters.ALL,
self.message_handler))

```

```

def is_valid_phone(self, phone: str) -> bool:
    return ((phone.startswith("+7") and len(phone) == 12) or
            (phone.startswith("8") and len(phone) == 11) or
            (phone.startswith("7") and len(phone) == 11))

```

```

def generate_payment_link(self, recipient: str, amount: str, details: str) -> str:
    base_url = "https://your-payment-gateway.com/create_payment"
    return
f' {base_url}?recipient={quote(recipient)}&amount={quote(amount)}&details={q
uote(details)}'

```

```

def extract_usernames(self, text: str) -> List[str]:
    return re.findall(r'@\w+', text)

```

```

def validate_amount(self, amount: str) -> bool:
    if not amount or amount.count('.') > 1:
        return False

```

```

    parts = amount.split('.')
    if len(parts) > 1 and len(parts[1]) > 2:
        return False

```

```

    cleaned = amount.replace('.', '')
    return cleaned.isdigit()

```

```

    async def start_command(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):

```

```

        chat_id = update.message.chat_id

```

```

        if self.db.is_user_registered(chat_id):
            user_data = self.db.get_user(chat_id)
            await update.message.reply_text(
                f'С возвращением, {user_data['name']}!\n"
                f'Ваш username: {user_data['username']}'
            )

```

```

            await self.show_main_menu(update.message)
        else:

```

```

            keyboard = [[KeyboardButton("Отправить номер телефона",
request_contact=True)]]

```

```

        reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True)
        await update.message.reply_text(
            "Добро пожаловать в Паука! Для регистрации поделитесь номером
            телефона:",
            reply_markup=reply_markup
        )

```

```

    async def debug_command(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):

```

```

        """Команда для отладки - показывает всех пользователей"""

```

```

        chat_id = update.message.chat_id
        user_data = self.db.get_user(chat_id)

```

```

        all_users = self.db.get_all_users()
        debug_info = f"""

```

Отладочная информация:

Ваш ID: {chat_id}

Ваши данные: {user_data if user_data else 'Не зарегистрирован'}

```

Все пользователи в базе ({len(all_users)}):
"""

```

```

        for uid, user in all_users.items():
            debug_info += f"- {uid}: {user}\n"

```

```

        await update.message.reply_text(debug_info)

```

```

    async def help_command(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):

```

```

        help_text = """

```

Команды бота:

/start - Начать работу

/help - Показать справку

/debug - Отладочная информация

Функции:

- Отправка денег

- Запрос денег

- Управление контактами

Для начала используйте /start

```

"""

```

```

        await update.message.reply_text(help_text)

```

```

async def show_main_menu(self, message):
    keyboard = [
        [
            InlineKeyboardButton("Отправить деньги",
callback_data="send_money"),
            InlineKeyboardButton("Запросить деньги",
callback_data="request_money")
        ],
        [
            InlineKeyboardButton("Мой профиль", callback_data="my_profile"),
            InlineKeyboardButton("Помощь", callback_data="help")
        ]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await message.reply_text("Главное меню - Выберите действие:",
reply_markup=reply_markup)

    async def callback_handler(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        query = update.callback_query
        await query.answer()

        chat_id = query.message.chat_id
        data = query.data

        if data == "send_money":
            await self.start_send_money(chat_id, query)
        elif data == "request_money":
            await self.start_request_money(chat_id, query)
        elif data == "no_message_send":
            await self.handle_no_message_send(chat_id, query)
        elif data == "confirm_send_send":
            await self.confirm_send_money(chat_id, query, context)
        elif data == "change_send_send":
            await self.change_send_money(chat_id, query)
        elif data == "no_message":
            await self.handle_no_message_request(chat_id, query)
        elif data == "confirm_send":
            await self.confirm_request_money(chat_id, query, context)
        elif data == "change_send":
            await self.change_request_money(chat_id, query)
        elif data.startswith("confirm_phone:"):
            await self.confirm_phone_registration(chat_id, query)
        elif data == "request_manual_phone":
            await self.request_manual_phone(chat_id, query)

```

```

elif data == "my_profile":
    await self.show_profile(chat_id, query)
elif data == "help":
    await query.message.reply_text("Используйте /help для получения справки")

```

```

async def start_send_money(self, chat_id: int, query):
    self.send_money_data[chat_id] = {
        'state': SEND_MONEY_STATES['WAITING_FOR_RECIPIENT'],
        'recipient': "",
        'amount': "",
        'message': "",
        'details': ""
    }
    await query.message.reply_text("Введите username получателя (например: @username):")

```

```

async def start_request_money(self, chat_id: int, query):
    self.request_money_data[chat_id] = {
        'state': REQUEST_MONEY_STATES['WAITING_FOR_RECIPIENT'],
        'payer': "",
        'amount': "",
        'message': "",
        'details': ""
    }
    await query.message.reply_text("Введите username плательщика/ов (например: @username1 @username2):")

```

```

async def handle_no_message_send(self, chat_id: int, query):
    if chat_id in self.send_money_data:
        self.send_money_data[chat_id]['message'] = ""
        self.send_money_data[chat_id]['state'] = SEND_MONEY_STATES['WAITING_FOR_CONFIRMATION']
        await self.show_send_confirmation(chat_id, query)

```

```

async def handle_no_message_request(self, chat_id: int, query):
    if chat_id in self.request_money_data:
        self.request_money_data[chat_id]['message'] = ""
        self.request_money_data[chat_id]['state'] = REQUEST_MONEY_STATES['WAITING_FOR_CONFIRMATION']
        await self.show_request_confirmation(chat_id, query)

```

```

async def show_send_confirmation(self, chat_id: int, query):
    data = self.send_money_data[chat_id]
    text = (f"Проверьте данные перевода:\n\n"

```



```

        f"Получатель: {data['recipient']}\n"
        f"Сумма: {data['amount']}\n")

    if data['message']:
        text += f"Сообщение: {data['message']}\n"

    keyboard = [
        [
            InlineKeyboardButton("Подтвердить",
callback_data="confirm_send_send"),
            InlineKeyboardButton("Изменить", callback_data="change_send_send")
        ]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await query.message.reply_text(text, reply_markup=reply_markup)

async def show_request_confirmation(self, chat_id: int, query):
    data = self.request_money_data[chat_id]
    text = (f"Проверьте данные запроса:\n\n"
        f"Плательщик: {data['payer']}\n"
        f"Сумма: {data['amount']}\n")

    if data['message']:
        text += f"Сообщение: {data['message']}\n"

    keyboard = [
        [
            InlineKeyboardButton("Подтвердить", callback_data="confirm_send"),
            InlineKeyboardButton("Изменить", callback_data="change_send")
        ]
    ]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await query.message.reply_text(text, reply_markup=reply_markup)

    async def confirm_send_money(self, chat_id: int, query, context:
ContextTypes.DEFAULT_TYPE):
        if chat_id not in self.send_money_data:
            return

        data = self.send_money_data[chat_id]
        recipient_input = data['recipient']

        recipient_username = recipient_input[1:] if recipient_input.startswith('@') else
recipient_input

```

```

print(f'DEBUG: Поиск пользователя '{recipient_username}''')

if self.db.is_user_registered_by_username(recipient_username):
    recipient_chat_id = self.db.get_chat_id_by_username(recipient_username)

    if recipient_chat_id:
        notification = (f'Вам перевод!\n\n'
                        f'От: @{query.message.chat.username}\n'
                        f'Сумма: {data['amount']}\n')

        if data['message']:
            notification += f'Сообщение: {data['message']}\n'

        await context.bot.send_message(recipient_chat_id, notification)

        payment_link = self.generate_payment_link(
            data['recipient'], data['amount'], "payment_details"
        )

        await query.message.reply_text(
            f'Перевод отправлен!\n\n'
            f'Ссылка для оплаты: {payment_link}\n'
            f'Скоро эта ссылка станет рабочей!'
        )
    else:
        await query.message.reply_text("Не удалось найти получателя")
    else:
        await query.message.reply_text(f'Пользователь {recipient_input} не
зарегистрирован в системе')

self.send_money_data.pop(chat_id, None)
await self.show_main_menu(query.message)

async def confirm_request_money(self, chat_id: int, query, context:
ContextTypes.DEFAULT_TYPE):
    if chat_id not in self.request_money_data:
        return

    data = self.request_money_data[chat_id]
    usernames = self.extract_usernames(data['payer'])

    success_count = 0
    failed_users = []

    for username in usernames:

```

```

clean_username = username[1:] if username.startswith('@') else username

print(f'DEBUG: Поиск плательщика '{clean_username}''')

if self.db.is_user_registered_by_username(clean_username):
    payer_chat_id = self.db.get_chat_id_by_username(clean_username)

    if payer_chat_id:
        request_msg = (f'Запрос на перевод\n\n'
                        f'От: @{query.message.chat.username}\n'
                        f'Сумма: {data['amount']}\n')

        if data['message']:
            request_msg += f'Сообщение: {data['message']}\n'

        try:
            await context.bot.send_message(payer_chat_id, request_msg)
            success_count += 1
        except Exception:
            failed_users.append(f'{username} (ошибка отправки)')
        else:
            failed_users.append(f'{username} (не найден)')
    else:
        failed_users.append(f'{username} (не зарегистрирован)')

if success_count > 0:
    result_msg = f'Запросы отправлены {success_count} пользователям\n'
else:
    result_msg = "Не удалось отправить запросы\n"

if failed_users:
    result_msg += "\nНе удалось отправить:\n" + "\n".join(failed_users)

await query.message.reply_text(result_msg)

self.request_money_data.pop(chat_id, None)
await self.show_main_menu(query.message)

async def change_send_money(self, chat_id: int, query):
    self.send_money_data.pop(chat_id, None)
    await query.message.reply_text("Введите username получателя (например: @username):")
    self.send_money_data[chat_id] = {
        'state': SEND_MONEY_STATES['WAITING_FOR_RECIPIENT'],
        'recipient': '',

```

```

        'amount': "",
        'message': "",
        'details': ""
    }

    async def change_request_money(self, chat_id: int, query):
        self.request_money_data.pop(chat_id, None)
        await query.message.reply_text("Введите username плательщика/ов
(например: @username1 @username2):")
        self.request_money_data[chat_id] = {
            'state': REQUEST_MONEY_STATES['WAITING_FOR_RECIPIENT'],
            'payer': "",
            'amount': "",
            'message': "",
            'details': ""
        }

    async def confirm_phone_registration(self, chat_id: int, query):
        if chat_id in self.user_phones:
            phone = self.user_phones[chat_id]
            username = f"@{query.message.chat.username}" if
query.message.chat.username else "user"
            name = f"{query.message.chat.first_name} {query.message.chat.last_name
or ""}.strip()"

            if self.db.save_user(chat_id, username, name, phone):
                self.user_phones.pop(chat_id, None)
                await query.edit_message_text("Регистрация завершена! Ваши данные
сохранены.")
                await self.show_main_menu(query.message)
            else:
                await query.edit_message_text("Ошибка при сохранении данных")
            else:
                await query.answer("Данные не найдены")

    async def request_manual_phone(self, chat_id: int, query):
        await query.message.reply_text("Пожалуйста, введите ваш номер телефона
вручную:")
        await query.answer("Введите номер телефона вручную")

    async def show_profile(self, chat_id: int, query):
        user_data = self.db.get_user(chat_id)
        if user_data:
            profile_text = (
                "Ваш профиль\n\n"

```

```

        f"ID: {chat_id}\n"
        f"Телефон: {user_data['phone']}\n"
        f"Имя: {user_data['name']}\n"
        f"Username: {user_data['username']}"
    )
    await query.message.reply_text(profile_text)
else:
    await query.message.reply_text("Профиль не найден. Используйте /start
для регистрации")

    async def message_handler(self, update: Update, context:
ContextTypes.DEFAULT_TYPE):
        if not update.message:
            return

        chat_id = update.message.chat_id
        text = update.message.text or ""

        if update.message.contact and not self.db.is_user_registered(chat_id):
            await self.handle_contact_registration(update)
            return

        if chat_id in self.send_money_data:
            await self.handle_send_money_states(update, chat_id, text)
            return

        if chat_id in self.request_money_data:
            await self.handle_request_money_states(update, chat_id, text)
            return

        if not self.db.is_user_registered(chat_id):
            await self.handle_manual_phone_input(update, chat_id, text)
            return

        if text.startswith('/'):
            return

            await update.message.reply_text("Используйте кнопки меню для
взаимодействия с ботом")

    async def handle_contact_registration(self, update: Update):
        chat_id = update.message.chat_id
        contact = update.message.contact

```

```

        username = f"@{update.message.chat.username}" if
update.message.chat.username else "user"
        name = f"{update.message.chat.first_name or ''}
{update.message.chat.last_name or ''}".strip()
        phone = contact.phone_number

        if self.db.save_user(chat_id, username, name, phone):
            await update.message.reply_text(
                f"Регистрация завершена!\n\n"
                f"Добро пожаловать, {name}!\n"
                f"Ваш username: {username}\n"
                f"Ваш номер: {phone}",
                reply_markup=None
            )
            await self.show_main_menu(update.message)
        else:
            await update.message.reply_text("Ошибка при сохранении данных.
Попробуйте еще раз.")

    async def handle_manual_phone_input(self, update: Update, chat_id: int, text:
str):
        if self.is_valid_phone(text):
            self.user_phones[chat_id] = text
            username = f"@{update.message.chat.username}" if
update.message.chat.username else "user"
            name = f"{update.message.chat.first_name or ''}
{update.message.chat.last_name or ''}".strip()

            keyboard = [
                [
                    InlineKeyboardButton("Да", callback_data=f"confirm_phone:{text}"),
                    InlineKeyboardButton("Нет", callback_data="request_manual_phone")
                ]
            ]
            reply_markup = InlineKeyboardMarkup(keyboard)

            await update.message.reply_text(f"Ваш номер: {text}\nВерно?",
reply_markup=reply_markup)
        else:
            await update.message.reply_text("Неверный формат номера. Пример:
+79991234567 или 89991234567")

    async def handle_send_money_states(self, update: Update, chat_id: int, text: str):
        data = self.send_money_data[chat_id]

```

```

if data['state'] == SEND_MONEY_STATES['WAITING_FOR_RECIPIENT']:
    if not text.startswith('@'):
        await update.message.reply_text("Username должен начинаться с @")
        return

    data['recipient'] = text
    data['state'] = SEND_MONEY_STATES['WAITING_FOR_AMOUNT']
    await update.message.reply_text("Введите сумму перевода (например:
100.50):")

elif data['state'] == SEND_MONEY_STATES['WAITING_FOR_AMOUNT']:
    if not self.validate_amount(text):
        await update.message.reply_text("Неверный формат суммы. Пример:
100.50")
        return

    data['amount'] = text
    data['state'] = SEND_MONEY_STATES['WAITING_FOR_MESSAGE']

    keyboard = [[InlineKeyboardButton("Без сообщения",
callback_data="no_message_send")]]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await update.message.reply_text("Введите сообщение для получателя:",
reply_markup=reply_markup)

elif data['state'] == SEND_MONEY_STATES['WAITING_FOR_MESSAGE']:
    data['message'] = text
    await self.show_send_confirmation(chat_id, update)

async def handle_request_money_states(self, update: Update, chat_id: int, text:
str):
    data = self.request_money_data[chat_id]

    if data['state'] ==
REQUEST_MONEY_STATES['WAITING_FOR_RECIPIENT']:
        usernames = self.extract_usernames(text)
        if not usernames:
            await update.message.reply_text("Укажите хотя бы одного пользователя
с @")
            return

        data['payer'] = text
        data['state'] = REQUEST_MONEY_STATES['WAITING_FOR_AMOUNT']
        await update.message.reply_text("Введите сумму (например: 100.50):")

```

```

elif data['state'] ==
REQUEST_MONEY_STATES['WAITING_FOR_AMOUNT']:
    if not self.validate_amount(text):
        await update.message.reply_text("Неверный формат суммы. Пример:
100.50")
    return

    data['amount'] = text

data['state'] =
REQUEST_MONEY_STATES['WAITING_FOR_MESSAGE']

    keyboard = [[InlineKeyboardButton("Без сообщения",
callback_data="no_message")]]
    reply_markup = InlineKeyboardMarkup(keyboard)
    await update.message.reply_text("Введите сообщение для плательщика:",
reply_markup=reply_markup)

elif data['state'] ==
REQUEST_MONEY_STATES['WAITING_FOR_MESSAGE']:
    data['message'] = text
    await self.show_request_confirmation(chat_id, update)

def run(self):
    print("Бот запущен и готов к работе!")
    print("Используйте /start в Telegram для начала")
    print("Для отладки используйте /debug")
    self.application.run_polling()

def main():
    bot_token = "8116100305:AAHP9UWfnHFNxy3-VxjmrzBOAYf1Mk0P9rw"

    if not bot_token or len(bot_token) < 20:
        print("ОШИБКА: Неверный токен!")
        print("Проверьте токен бота")
        return

    try:
        bot = MoneyBot(bot_token)
        bot.run()
    except Exception as e:
        print(f"Не удалось запустить бота: {e}")

if __name__ == "__main__":
    main()

```


ss.ru

DEBUG: Загружено 1 пользователей

DEBUG: Пользователь 991498468: {'username': '@murkece', 'name': 'murkece', 'phone': '79167940301'}

Бот запущен и готов к работе!

Используйте /start в Telegram для начала

Для отладки используйте /debug

□